End-to-End RTL-to-GDSII Flow for an FSM Based Management Controller Design

Submitted By:

Rishank Jain (22102010) A1
Lakshya Kandpal (22102015) A1

Under the Guidance of:
Dr. Shamim Akhter
Associate Professor & Associate Dean
(Alumni & Students Affairs)
Department Of Electronics And
Communication Engineering

Motivation - Why This Project

We often hear about **chips powering smartphones, cars, AI, and supercomputers**, but rarely see **how these chips are actually designed and built**.
Behind every chip is a **complex VLSI backend flow**, transforming abstract logic into physical layouts ready for fabrication.

As students of Electronics, we wanted to go **beyond textbooks** and understand:
“What really happens from writing RTL code to generating the final GDSII file that goes to the foundry?”

This project gave us a chance to **bridge theory and industry practice**, working with **tools** used by chip designers worldwide.

From Classroom Learning to Hands-On Practice

•In our coursework we studied subjects like:
•**VLSI Design** – principles of backend design flows
•**Digital Circuit Design** – fundamentals of logic implementation
•**VLSI Fabrication** – how chips are physically built
•**Digital Hardware Design** – RTL and hardware-level thinking

•These subjects **taught us the concepts of how chips are designed**, but mostly at a **theoretical level**.

We wanted to go **beyond lectures and textbooks** to truly understand:

“How do all these steps come together in a real design project?”

Problem Statement

The problem is to **systematically learn and validate the complete RTL to GDSII flow** using a simple and well-defined design first (Covered in Major-1), and then extend the same methodology to a **realistic FSM-based Management Controller**.

Two-Phase Project Approach

Phase 1: Backend Flow Understanding Using Full Adder (MAJOR-1)

- Implement the complete **RTL to GDSII flow** on a simple combinational circuit.
- Focus on understanding **EDA tool behavior**, design stages, and report generation steps which can help in phase 2.
- Perform synthesis, floorplanning, power planning, placement, routing, and GDSII generation.
- Build a strong foundation without functional complexity.

Two-Phase Project Approach

Phase 2: FSM-Based Management Controller Implementation

- Apply the same backend methodology to a **multi-state FSM-based design**.
- Handle increased design complexity, including **sequential logic and clock distribution**.
- Perform timing closure, power optimization, and final sign-off checks.
- Generate a **manufacturing-ready GDSII layout** for a realistic controller design.
- Interpret the various reports that will be generated when clock is involved in the circuit.

Tools Used in the Project

We used **Cadence EDA Suite**, which provides industry-standard tools for every stage of the backend flow:

1. **NCLaunch** – RTL coding & simulation
2. **Genus** – Logic synthesis
3. **Innovus** – Floorplanning, power planning, placement, CTS, routing, RC extraction
4. **Tempus** – Static timing analysis and sign-off

What is done till now?

- **Conceptual Understanding**
  - Studied the complete RTL-to-GDSII backend design flow in a structured, stage-by-stage manner.
  - Understood the role, inputs, outputs, and dependencies of each physical design stage, from RTL simulation to GDSII generation.
  - Gained clarity on how timing, power, and area constraints propagate across the backend flow and why does constraints like clock play an important role in modern VLSI designs.
- **Reference Study**
  - Referred to Cadence tool manuals, laboratory procedure documents, and standard VLSI backend tutorials.
  - Studied academic literature and technical resources related to synthesis, placement, routing, and timing analysis.
  - Prepared a step-by-step execution reference for RTL simulation and initial backend stages.

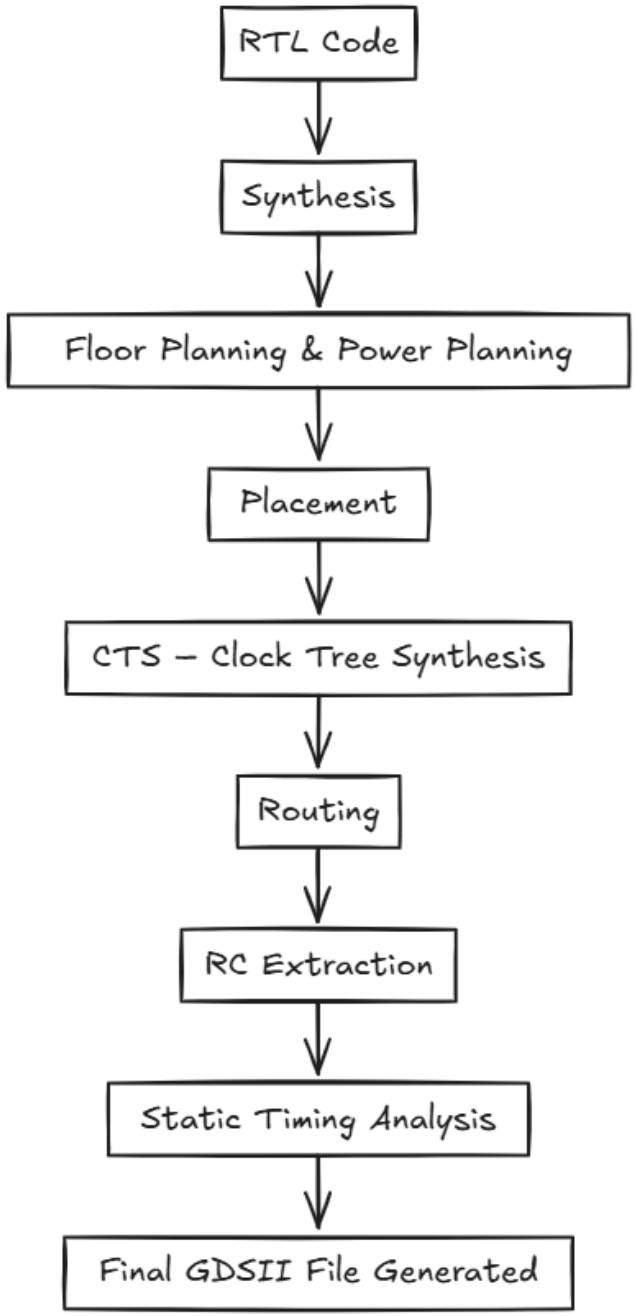
What is done till now?

- **RTL Design and Functional Verification**
  - Written Verilog RTL code for a Full Adder as the Phase 1 design.
  - Developed a dedicated testbench to verify all possible input combinations.
  - Performed functional simulation using NCLaunch and SimVision to confirm logical correctness.
- **Logic Synthesis Practice**
  - Performed logic synthesis using Cadence Genus to convert RTL into a gate-level netlist.
  - Various reports like area, cell usage and SDC (Static Design Constraints) were created.
  - Learned how to generate synthesis reports related to timing, area, cell utilization, and power consumption.

Work Flow:

- RTL Code
- Synthesis
- Floor Planning and Power Planning
- Placement
- CTS (Clock Tree Synthesis)
- Routing
- RC Extraction
- Static Time Analysis
- Final GDSll file generated.

Work Flow:


RTL Code
Synthesis
Floor Planning & Power Planning
Placement
CTS – Clock Tree Synthesis
Routing
RC Extraction
Static Timing Analysis
Final GDSII File Generated

RTL Code Definition:

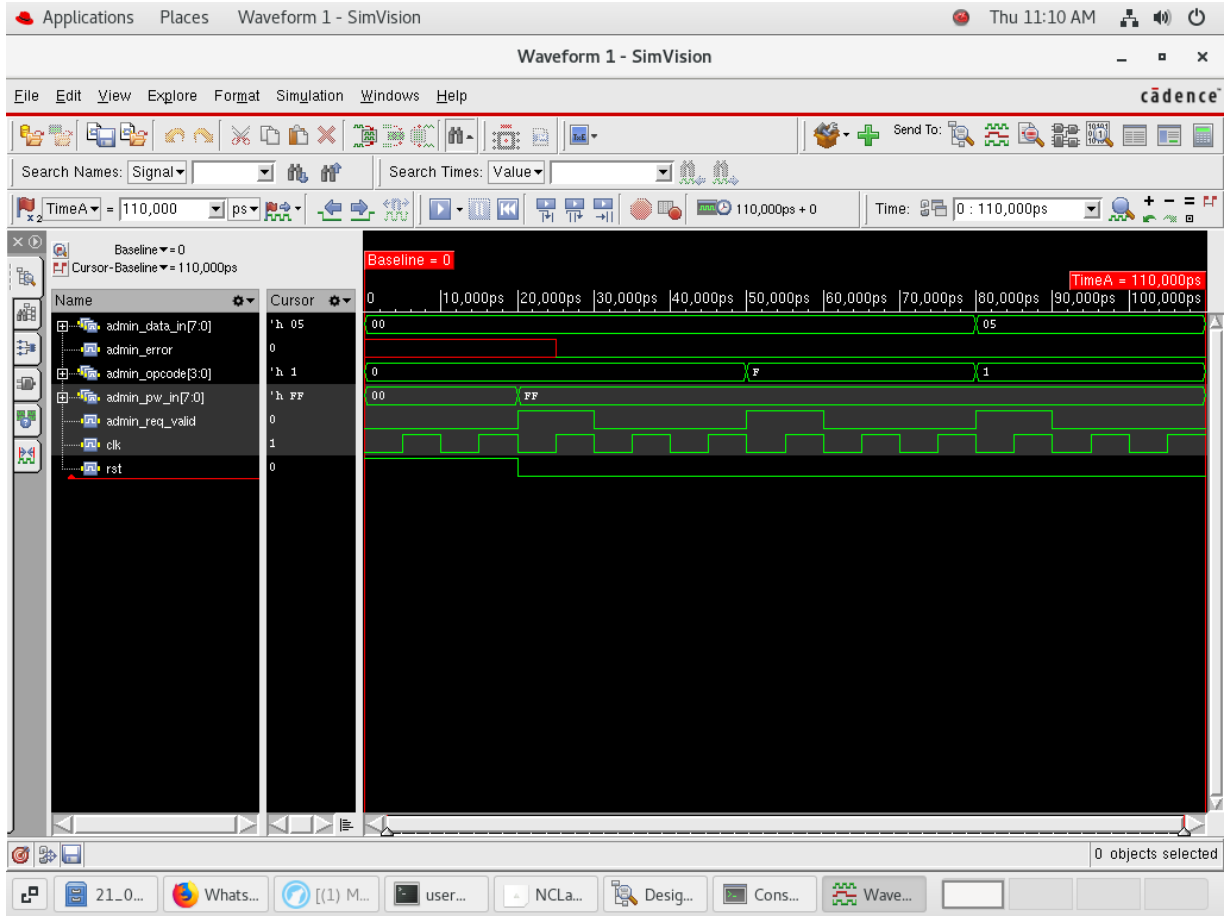
Register Transfer Level (RTL) code is a hardware description of a digital circuit written in languages such as Verilog or VHDL. It describes the behavior of a system in terms of the flow of data (transfers) between registers and the logical operations performed on that data, synchronized by a clock.

- **Explained Simply:**
  Imagine you are describing how a factory works. You don't draw the full machines with nuts and bolts, instead, you explain:
  - What each worker (register) does,
  - How the raw material (data) moves from one worker to another,
  - And when they should do their job (the clock).
  - That description is the **RTL code**. It's not the final physical machine yet - it's more like a blueprint that explains the **logic and flow of work**.

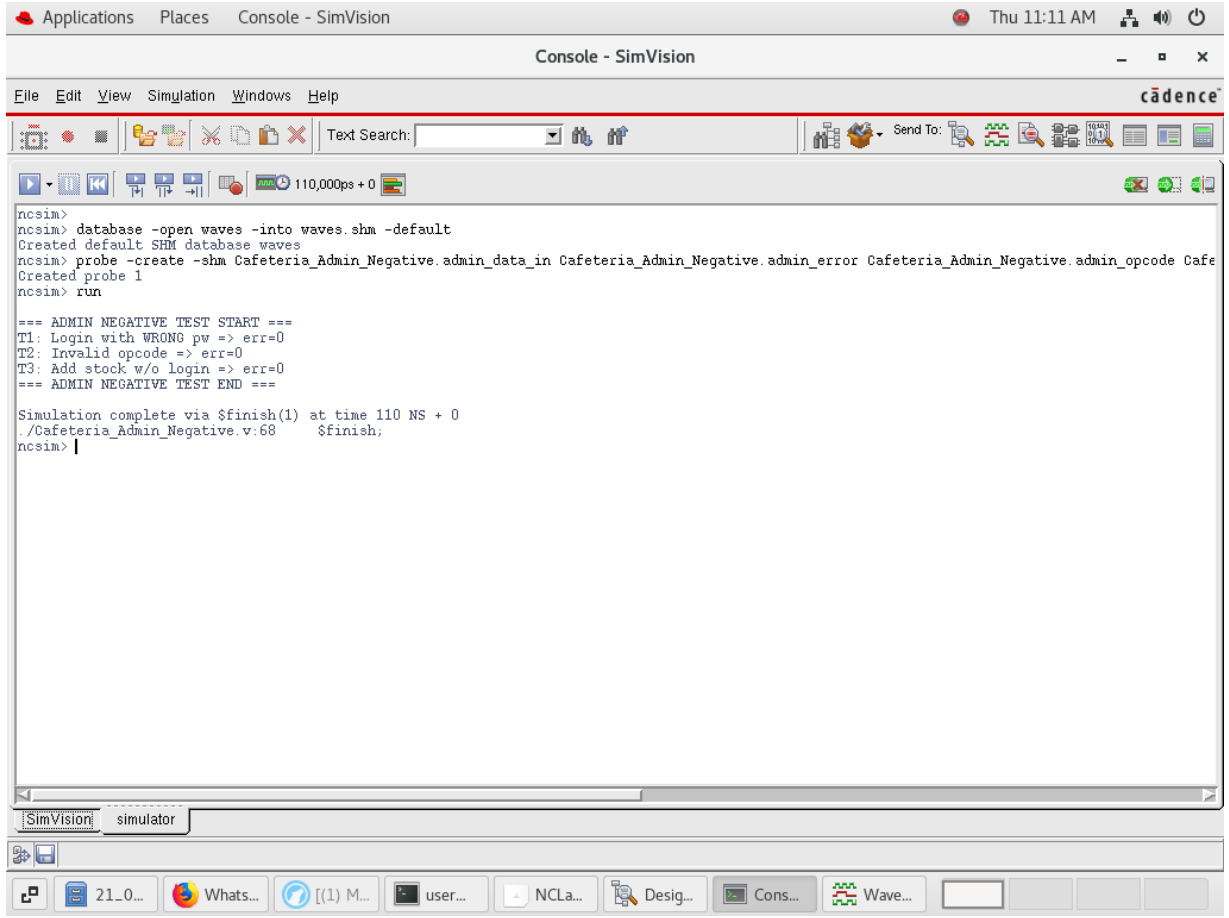
RTL Code – Why is it Needed?

- **The Problem in Chip Design:**
  - Modern chips are extremely complex, containing millions or billions of transistors.
  - Designing directly at the transistor or gate level is impractical and error-prone.
  - Without a higher-level description, it becomes impossible to verify the logic or make changes quickly.
- **How RTL Solves the Problem:**
  - RTL provides a **human-readable abstraction** of the circuit using Verilog/VHDL.
  - It allows designers to describe **what the circuit should do** instead of manually wiring gates.
  - Enables **simulation and verification** early in the design flow to catch logic bugs before synthesis.
  - Supports **reusability and scalability** – blocks like adders, multipliers, controllers can be reused in multiple projects.
  - Creates a solid foundation for automation: once RTL is written and verified, tools can translate it into gates, layouts, and finally silicon

RTL Steps (Admin Negative)



RTL Code Steps



Synthesis – Definition

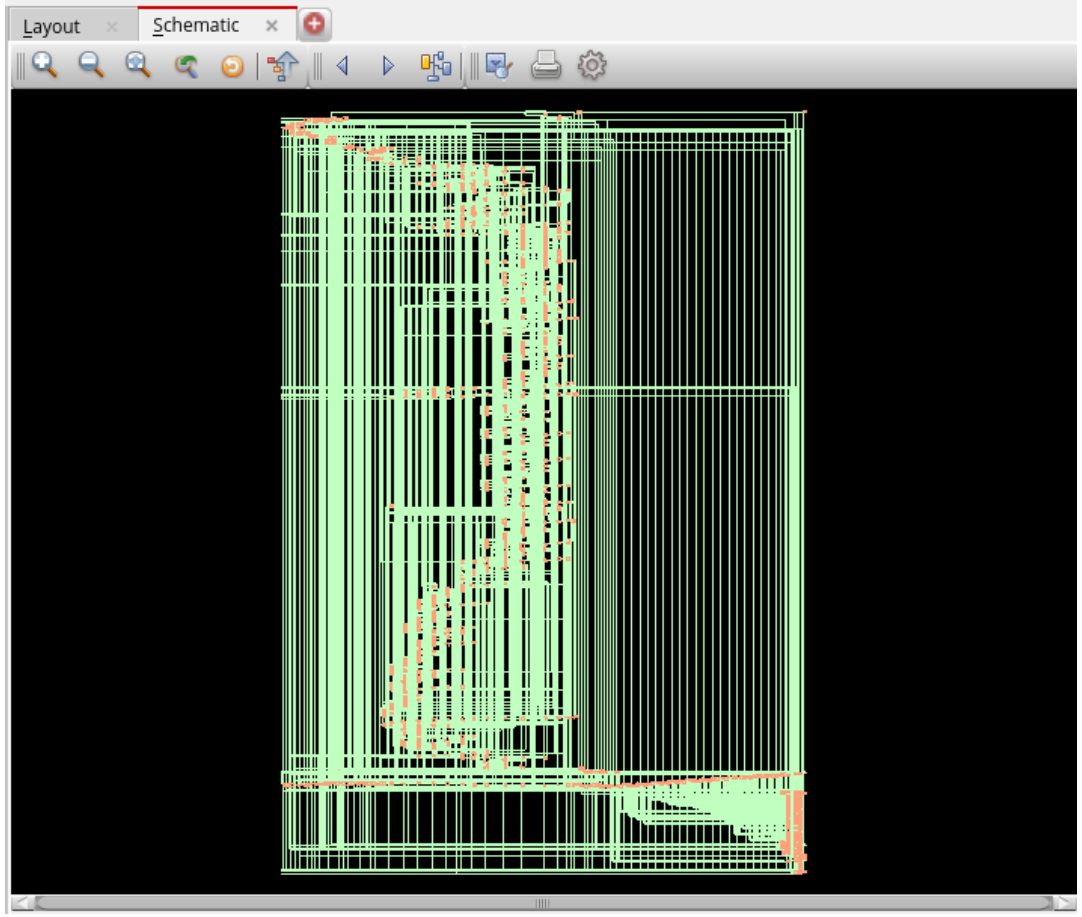
Synthesis is the process of converting an RTL description (Verilog/VHDL) into a **gate-level netlist** using the standard cell library of a specific technology node **(In our case, we use 180nm library)**. The tool maps the high-level logic into actual logic gates and flip-flops while optimizing for timing, area, and power.

- **Explained Simply:**
  Think of RTL code as a recipe written in words. You know what dish you want, but not the exact steps to cook it.
  Synthesis is like a **chef** who takes that recipe and translates it into the real cooking steps - chopping, frying, baking - using actual ingredients from the kitchen.
  In the same way, synthesis takes your logical description and **builds it from real gates and flip-flops** that exist in the technology library.

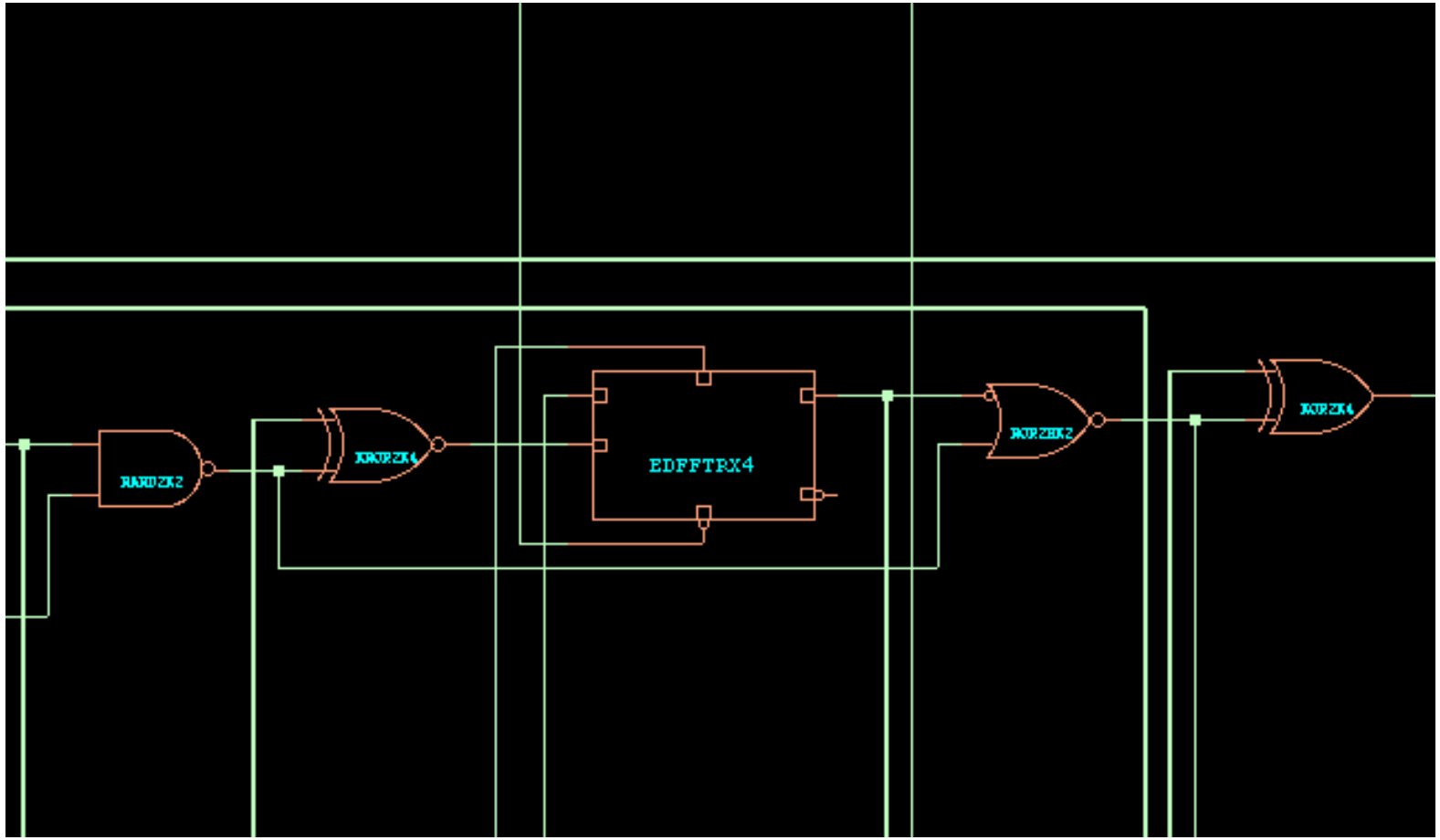
Synthesis – Why is it Needed?

- **The Problem in Chip Design:**
  - RTL is just a **behavioral description** - it cannot be fabricated directly.
  - Manually mapping RTL to millions of logic gates would be **impossible and error-prone**.
  - Without synthesis, designers would struggle to meet strict **timing, area, and power** requirements.
- **How Synthesis Solves the Problem:**
  - **Automation:** Converts RTL into a **gate-level netlist** quickly and accurately.
  - **Optimization:** Balances trade-offs between **speed (timing), silicon area, and power**.
  - **Technology Mapping:** Adapts the RTL to a specific process node using the foundry's standard cells.
  - **Early Feedback:** Generates report on power so issues can be fixed before expensive backend stages.

Synthesis – Steps



Synthesis – Steps



Synthesis – Steps

Instance	Cells	Leakage (nW)	Internal (nW)	Net (nW)	Switching (nW)
Cafeteria	2263	1655.984	368059.051	83437.739	451496.790

Floorplanning – Definition

Floorplanning is the process of defining the **die/core area, aspect ratio, I/O pad locations, and macro/block placement** before detailed placement of standard cells. It acts as the **physical blueprint** of the integrated circuit.

- Imagine you're designing a new city. Before you build houses (standard cells), you must decide:
  - How big the city will be (die/core size),
  - Where the big buildings like malls and schools (macros/IPs) should go,
  - Where the roads and entry/exit gates (I/O pads) will be placed,
  - And how much open space to leave so people don't get stuck in traffic (utilization).
  - That's exactly what **floorplanning** does for a chip — it's like creating the city map before construction starts.

Floorplanning – Why is it Needed?

- **The Problem in Chip Design:**
  - Without a proper plan, the design may suffer from **congestion**, making routing nearly impossible.
  - Poor placement of macros/IP blocks can cause **long interconnect delays**, hurting performance.
  - Uneven distribution of logic leads to **hotspots** (areas consuming more power).
  - Lack of planning increases chances of **timing violations** and **manufacturability issues**.
- **How Floorplanning Solves the Problem:**
  - Defines **chip size and aspect ratio**, balancing area and performance.
  - Places **macros/IPs strategically** to minimize wirelength and congestion.
  - Organizes **I/O pads** for smooth connectivity with external signals.
  - Prepares for **power planning** by reserving space for power rings and stripes.

Floorplanning – Steps

- **Core Utilization = 0.8**,
  it means **80 percent of the core area is occupied by standard cells**, and **20 percent is left empty** for routing, buffers and optimization.

According to the procedure, we entered:

Aspect Ratio : 1

Core Utilization : 0.8

Power Planning – Definition

Power planning is the process of designing the **power distribution network (PDN)** — including rings and stripes — to ensure stable and uniform supply of power (VDD) and ground (VSS) across the entire chip, while preventing IR-drop and electromigration.

Explained Simply:
Imagine a city where electricity has to reach every house. If the wires are too thin or not well planned, some houses will get dim lights or even no power at all.
Power planning is like building a strong network of **main power lines (rings) and streets (stripes)** to make sure every building (cell) in the city (chip) gets reliable electricity.

Placement - Definition

Placement is the process of assigning **exact physical locations** to all standard cells inside the core area defined during floorplanning. It arranges cells in legal positions along placement rows, optimizing for timing, area, congestion, and power.

Explained Simply:
Imagine arranging desks in a big classroom. The teacher (clock) needs to reach every student (cell) on time, and students who work together should sit close to each other so they don't waste time passing notes (signals).
Placement is like deciding **where each desk goes** so that:

- Nobody is overlapping,
- The classroom isn't too crowded,
- And everyone can easily talk to the right partners.
- That's exactly what placement does in a chip — it decides the **seat** of every standard cell.

Placement - Why is it Needed?

- **The Problem in Chip Design:**
  - After synthesis, we only know which cells exist, but **not where they should go** on the chip.
  - If cells are placed randomly:
    - Wires become very long, causing **timing delays**.
    - Certain areas may get overcrowded, leading to **routing congestion**.
    - Power delivery may become uneven, creating **hotspots**.
  - Without structured placement, meeting timing and power targets becomes nearly impossible.
- **How Placement Solves the Problem:**
  - Places cells in legal rows without overlaps (legalization).
  - Groups cells that talk frequently closer together → **reduces wirelength**.
  - Uses **timing-driven placement** to ensure critical-path cells are placed optimally.
  - Distributes cells evenly to avoid congestion and enable smoother routing.
  - Balances placement density to support reliable **power delivery**.

Placement – Steps

- **Endcap (or boundary) cells are placed at the ends of standard cell** rows (left and right, and sometimes top and bottom boundaries of a block) to satisfy specific design rule check (DRC) requirements.
- Endcap **protect the gates of the standard cells placed near the boundary from potential damage** during the chip manufacturing process.
- Well tap cells (or tap cells) are used to prevent a phenomenon called **latch-up** in CMOS designs, which is a potential short circuit between power and ground rails.

Clock Tree Synthesis (CTS)

Clock Tree Synthesis (CTS) is the process of building a **balanced clock distribution network** by inserting buffers, inverters, and routing paths so that the clock signal reaches all sequential elements (flip-flops and latches) with controlled **skew** and **latency**.

Explained Simply:
Imagine a school where the bell tells every student when class starts. If some students hear the bell later than others, the whole system falls into chaos. CTS is like installing **loudspeakers** all over the school so that everyone hears the bell **at nearly the same time**. It uses amplifiers (buffers) and wires to make sure the sound (clock) is strong and synchronized everywhere.

Clock Tree Synthesis (CTS) – Why is it Needed?

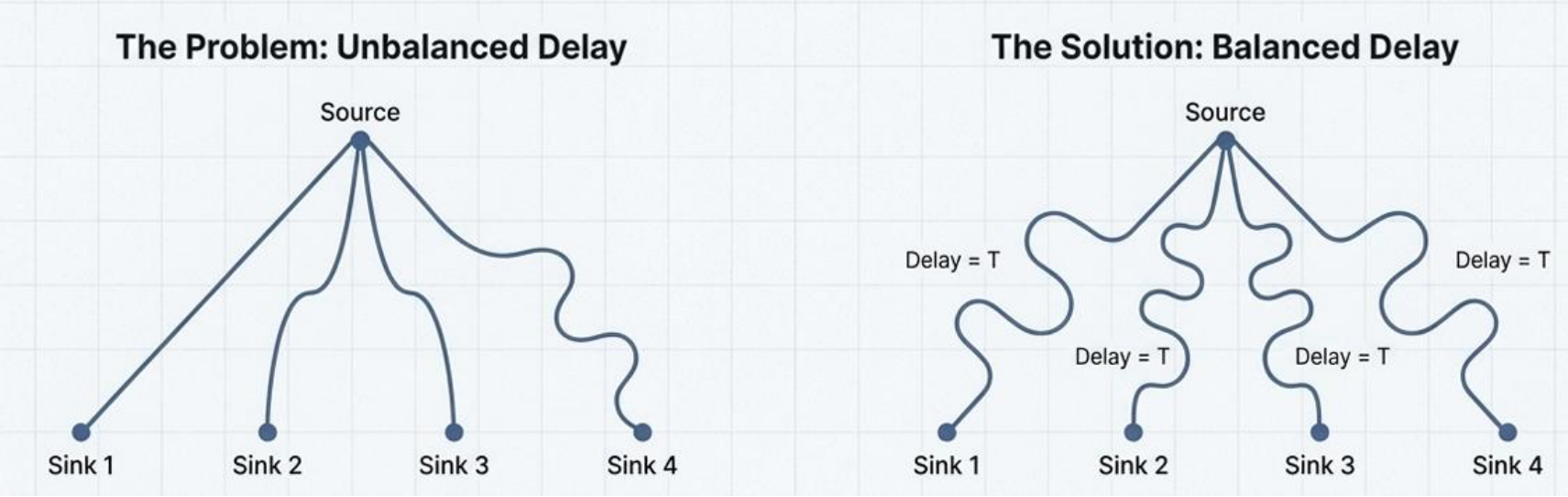
- **The Problem in Chip Design:**
  - The clock is the **heartbeat of the chip**, synchronizing millions of flip-flops.
  - If the clock doesn't arrive at all flip-flops at nearly the same time:
    - **Clock Skew**: Some flip-flops capture data too early or too late.
    - **Clock Latency**: Clock may arrive too late, failing to meet timing.
  - Without CTS, timing closure becomes nearly impossible, leading to **setup and hold violations**.
  - A poorly designed clock network wastes power, as the clock is always toggling, even in idle circuits.
- **How CTS Solves the Problem:**
  - Inserts **buffers/inverters** to balance clock arrival times across the chip.
  - Minimizes **clock skew** by ensuring clock paths are matched.
  - Controls **insertion delay (latency)** to fit timing requirements.
  - Supports **useful skew**, intentionally delaying some paths to improve setup/hold balance.
  - Enables **clock gating**, which reduces switching in idle regions and saves power.

Clock Tree Synthesis (CTS) – Why is it Needed?

Theoretically what are the strategies to reduce SKEW?

Two main strategies:

- **Place all clock inputs close together**
  - Not practical for large chips
- **Drive them from the same source & balance delays**
  - Most commonly used
  - Achieved using structured networks like trees or meshes



CTS: Implementation and Observations

- **Reason for Implementing CTS**
  - Clock Tree Synthesis was included to **maintain completeness of the standard RTL-to-GDSII backend flow**.
  - CTS is a mandatory stage in industrial backend design flows, especially for **sequential circuits** such as FSMs.

CTS: Implementation and Observations

- **Design Context in Phase 1**
  - The Phase 1 design is a **Full Adder**, which is a purely **combinational circuit**.
  - The design does **not contain any sequential elements** such as flip-flops, latches, or registers.
  - As a result, there is **no clock pin or clock net** present in the design.
- **CTS Execution Outcome**
  - CTS scripts were executed correctly within the Innovus environment.
  - The tool reported the **absence of clocked elements**, leading to no clock buffers or clock tree generation.
  - Since CTS operates only on clock paths driving sequential elements, **no visible clock tree was produced**.

CTS: Implementation and Observations

- **Key Learning from CTS Stage**
  - CTS is **functionally meaningful only for sequential designs** that require synchronized clock distribution.
  - In combinational circuits, CTS does not generate outputs, but its inclusion validates **methodological correctness**.
  - This observation reinforces the importance of CTS in the upcoming **FSM-based Management Controller**, where clock distribution will be critical.

Routing – Definition

Routing is the process of creating **physical interconnections** (wires and vias) between all standard cells, macros, and I/O pins according to the netlist, while following design rules (DRC) and optimizing for timing, power, and signal integrity.

Explained Simply:
Imagine you've placed all the desks (cells) in a classroom. Now you need to draw paths between them so students can pass notes (signals). These paths must be:

- Short enough so messages arrive quickly,
- Wide enough so notes don't get jammed (no congestion),
- And follow the classroom rules (no overlapping lines).
- That's what **routing** does in a chip — it builds the actual "roads" (wires) so that all parts of the circuit can talk to each other.

Routing – Why is it Needed?

- **The Problem in Chip Design:**
  - After placement and CTS, cells and clock buffers are positioned but **not physically connected**.
  - If signals aren't routed properly:
    - Data won't travel between cells → the chip won't function.
    - Long or overlapping wires create **timing violations**.
    - Overcrowded routes cause **congestion** and may violate design rules (DRC).
    - Crosstalk between wires can cause **noise and glitches**.
  - Power rails also need routing to ensure every cell receives stable supply.
- **How Routing Solves the Problem:**
  - **Global Routing:** Finds rough paths for all nets, balancing congestion and timing.
  - **Detailed Routing:** Creates exact wire and via geometries following design rules.
  - **Timing-Driven Routing:** Prioritizes critical paths to reduce delay and meet frequency targets.
  - Ensures that **every signal and power net** is physically connected and manufacturable.

RC Extraction

RC Extraction is the process of calculating the **resistance (R)** and **capacitance (C)** of all wires and vias in the routed design. These parasitic are captured in a particular file format and used for accurate timing, power, and signal integrity analysis.

- **Explained Simply:**
  Imagine water flowing through a network of pipes. The longer and thinner the pipes, the harder it is for water to flow (resistance). If pipes run close together, water pressure can leak between them (capacitance).
  RC Extraction is like measuring how much **friction (R)** and **leakage (C)** each pipe has, so we know how fast water can really flow. For chips, it tells us how signals will actually behave on real wires.

RC Extraction – Why is it Needed?

The Problem in Chip Design:

- After routing, wires are assumed ideal if parasitics are not considered.
- In reality, wires have **resistance (R)** and **capacitance (C)**, which cause:
  - **Delays** on signal paths → timing violations.
  - **Crosstalk noise** between nearby nets.
  - Increased **dynamic power consumption** due to higher capacitance.
- Without RC data, timing and power analysis would be inaccurate, leading to chips that fail in real silicon.

How RC Extraction Solves the Problem:

- Accurately measures parasitic **R and C** values for all interconnects.
- Produces files like **SPEF (Standard Parasitic Exchange Format)** for use in STA and power analysis.
- Enables precise **timing closure** by accounting for interconnect delay.
- Helps detect **signal integrity issues** like crosstalk-induced glitches.
- Provides realistic data for **power estimation** across the design.

RC Extraction – Reports Gathered

Reports Collected at RC Extraction Stage:

- **Power Report (Parasitic-aware):** More accurate power estimation, including interconnect effects.
- We can also get various other reports at this step like timing report.

Are These Reports Final?

- These reports are **close to final**, since they use actual routed geometries.

Static Timing Analysis (STA) – Definition

Static Timing Analysis (STA) is a method of verifying the **timing performance** of a digital circuit. It checks all possible paths in the design to ensure they meet **setup and hold time requirements** across all process, voltage, and temperature (PVT) corners.

Explained Simply:
Imagine a relay race where each runner must pass the baton within a fixed time. If someone runs too slow, the next runner gets delayed (setup violation). If someone runs too fast, the baton is passed before the next runner is ready (hold violation).
STA is like a stopwatch system that checks every possible path in the race, making sure the baton (data) always arrives at the right time for every runner (flip-flop).

Final GDSII

GDSII (Graphic Data System II) generation is the final step of the physical design flow where the verified layout is converted into the **industry-standard mask data format** (GDSII).

Explained Simply:
Imagine you've built a Lego model on your computer. To actually make it in real life, you need detailed blueprints that a factory can follow to cut the Lego blocks.
In chip design, the GDSII file is that **blueprint** — it contains every tiny detail (layers, shapes, and masks) needed to manufacture the chip. Sending this blueprint to the factory (the foundry) is called **tapeout**.

Final GDSII – Why is it Needed?

The Problem in Chip Design:

- Up to this point, the design exists only as a digital database inside EDA tools.
- The foundry cannot manufacture a chip from RTL, netlists, or files — it needs a **mask-level blueprint**.
- Without a clean, verified GDSII file, the risk of chip failure in silicon is extremely high.
- Any error at this stage can cause a **failed tapeout**, wasting millions of dollars.

How GDSII Solves the Problem:

- Converts the final verified layout into **mask data** (GDSII) that foundries use to build photomasks.
- Includes **all physical verification checks** to ensure manufacturability.
- Packages timing, power, and signal integrity sign-off reports along with layout.
- Serves as the **frozen golden version** of the design — no more changes are allowed.
- Officially hands off the design from engineers to the foundry for fabrication.